gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843
Tech Debt and V&V

Francisco Gomes de Oliveira Neto

(he/him)

Dorothy Vaughan

Headed the programming section of the Analysis and Computation Division (ACD) at NASA

To prepare for the introduction of computers in the 1960s, she taught herself and her entire staff the programming language of Fortran.

What will we learn today?

- What is **Technical Debt**? Why should we measure it?
- What are the **different types** of technical debt?
- What are the main **causes** of Technical Debt?
- How to **handle and mitigate** technical debt?
- What is **V&V**? What are **faults, failures and errors**?

© Francisco Oliveira Neto - 2018

Part 1:
Technical Debt

© Francisco Oliveira Neto - 2018

W. Cunningham, “The WyCash portfolio management system,” doi: 10.1145/157709.157715

Technical Debt

- TD is the **implied cost of reworking** a non-optimal design solution that was chosen due to lack of time or resources to implement a better solution.
  - Simple example: Using magic numbers instead of constants. Other code smells.

- The **debt metaphore** conveys that the development process pays interests by keeping to postpone these solutions.

© Francisco Oliveira Neto - 2018

W. Cunningham, “The WyCash portfolio management system,” doi: 10.1145/157709.157715

Technical Debt

An average of **30% up to 80%** of the development resources are **wasted** because of TD.

TD can be **deliberate**, if taken to speed up development.

© Francisco Oliveira Neto - 2018

Kruchten, Philippe, Robert L. Nord, and Ipek Ozkaya. "Technical debt: from metaphor to theory and practice." IEEE Software 6 (2012): 18-21

Technical Debt - Landscape

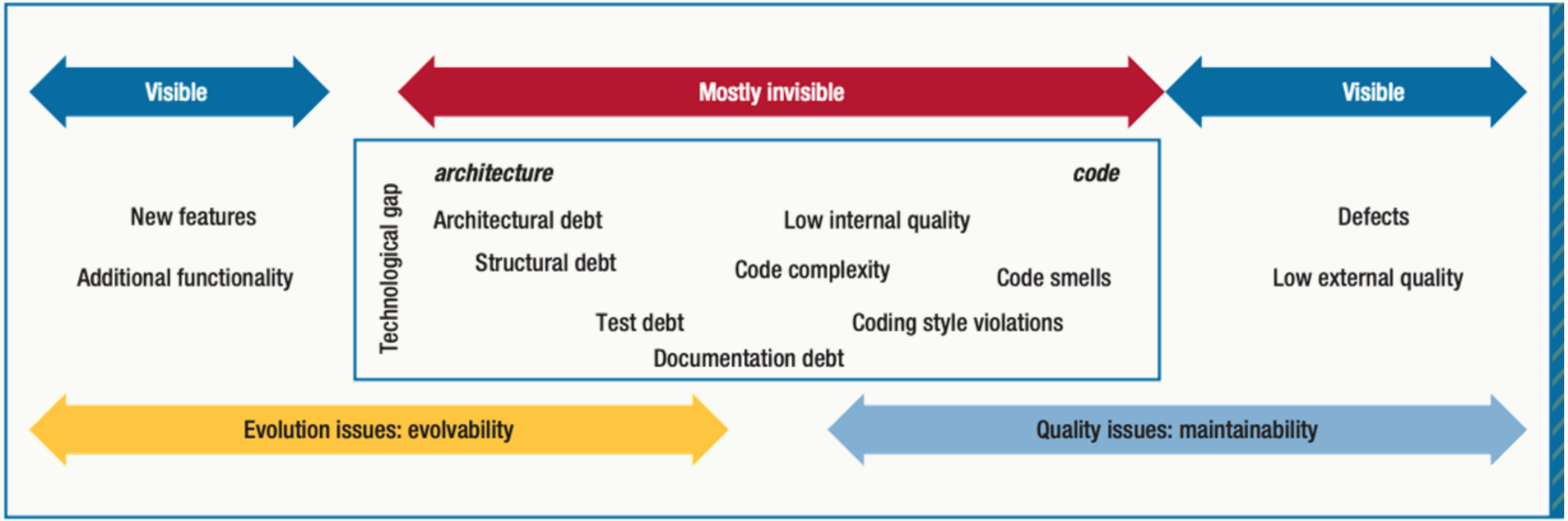


FIGURE 1. The technical debt landscape. On the left, evolution or its challenges; on the right, quality issues, both internal and external.

© Francisco Oliveira Neto - 2018

Technical Debt – Causes

- Developing and delivering **very rapidly**, with **no time** for proper design or to reflect on the longer term effect.

- **Lack of rigor** or systematic / automated testing.

© Francisco Oliveira Neto - 2018

Technical Debt – Causes

- Software companies are still immature at managing TD, using unsystematic approaches to handle even code-level debt.
- The whole purpose of TD research is to understand **when** TD should be **taken up**, **kept**, **avoided** or **repaid**.

- Technical Debt is worse in the context of **Machine Learning systems**.

© Francisco Oliveira Neto - 2018

Technical Debt – Key challenges

- **Dependency debt** is the key contributor to code complexity and TD.
- Static analysis is key in mitigating **data dependency**.

- **Technical Debt in ML systems** is a booming area of SE research.
  - Eroding abstractions and strict boundaries hinder “automated” checks.
  - Training and evolving learned models hinder adaptability of input.
  - Creating, integrating, and evolving components can lead to ”frozen” solutions.

© Francisco Oliveira Neto - 2018

Technical Debt – How to Handle

- The best way to handle Technical Debt is to **be aware of it**.
  - Sonarqube offers a toolkit of measures to avoid code debt.
- Keep a **backlog of debt items** and prioritise those items.
- Use TD to inform your decisions. Sometimes, “taking” the debt can be very beneficial for **short-term “profits”**.

© Francisco Oliveira Neto - 2018

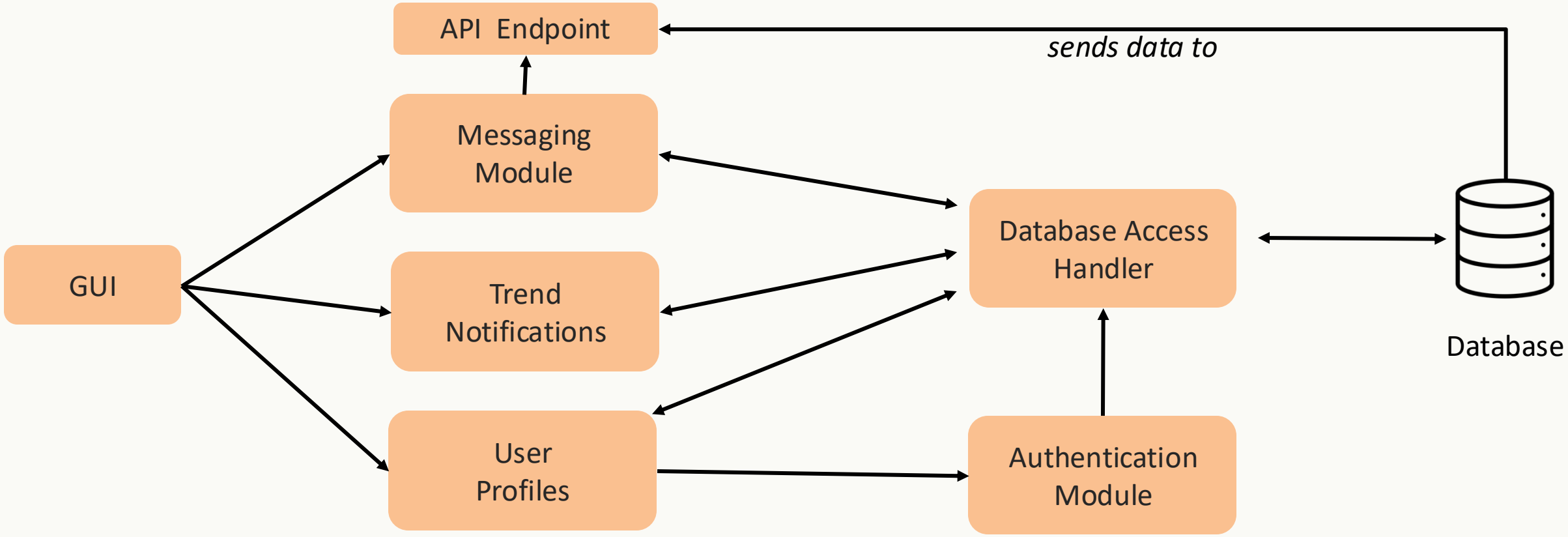
Exercise 1 – TrendNet System

- **Scenario:** Your team works on a social media app called **TrendNet**, which has been evolving for the past five years. Due to initial rapid growth and frequent feature additions, the artefacts have accumulated some technical debt. The following points outline the state of different artefacts:

- **Features:** The app supports user profiles, direct messaging, and real-time trend notifications.

© Francisco Oliveira Neto - 2018

TrendNet Architecture



© Francisco Oliveira Neto - 2018

Known issues:

- The messaging feature uses duplicated code across different modules.
- Some sections of the codebase still rely on an outdated framework version.
- The module handling trend notifications has grown increasingly complex, with nested conditional logic.
- The documentation for the core data models and API endpoints is outdated and lacks crucial details.
- The notification logic contains magic numbers for time intervals and user thresholds.
- All database interactions for the app go through a single module that has grown large.
- The authentication module was developed with old security practices and lacks modern encryption methods.
- While there are some unit tests, integration tests for key interactions between the messaging and notification modules are missing.

© Francisco Oliveira Neto - 2018

Exercise 1 – TrendNet System

1. **Classify the types of technical debt** found (e.g., code-level, architectural, test debt).

2. List steps you would take to **manage or mitigate this technical debt** before or during the implementation of the new feature.

© Francisco Oliveira Neto - 2018

Exercise 1: Answers

Q1: Classification of the Debt

- **Code-Level Debt:** Magic numbers, duplicated code, complex nested logic.
- **Architectural Debt:** Outdated framework, monolithic database access, legacy authentication module.
- **Test Debt:** Sparse unit tests, insufficient integration tests.
- **Documentation Debt:** Outdated or incomplete documentation.

Q2: How to handle:

- **Code-level:** Use metrics such as cyclomatic complexity, and static analysis tools.
- **Architectural:** Use metrics such as fan-in, fan-out to identify bottle necks.
- **Test Debt:** Ensure minimal test coverage
- **Documentation:** Enforce documentation checks when doing code reviews for merge requests.

© Francisco Oliveira Neto - 2018

Part 2:

V&V and Bugs

© Francisco Oliveira Neto - 2018

Validation and Verification (V&V)

- **Verification** is intended to check whether the product **meets** a set of **design specifications**.
  - This is often a process done within the development team.
- **Validation** is intended to ensure whether the product **meets** the operational **needs of the user.**
  - This is often a process done ”outside” the development team (with customer).

Verification

- The focus is whether the implemented software is **consistent with the specification.** Connected to a design or requirements **specification** (explicit or implicit).
- Verification activities rely on **collecting data** on the developed **artefacts** (code, architecture, environment infrastructure, tests, etc.) or during the **execution** of the software (e.g., log failures, measure crashes and robustness).

Validation

- Validation is **very hard** to do!
- Focuses on the product being used in the “real world”. Connected to the **actual user needs** (implicit or explicit).
- Even if the software created **complies with the specification**, we can **still fail the validation** of our software. For instance, in case we missed to capture some of the user’s needs in the specification.

Verification and Validation

Validation	Verification
Are we building the **right software?**	Are we building the **software right?**

V&V and Testing

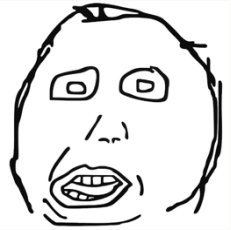
- We should not use those terms interchangebly.
- Testing is **one way** of doing V&V. One of the most **popular**.
  - Software Testing is a V&V activity done by **observing the software's behaviour**. Testing requires that the software is running.
- **Other** V&V activities are: Formal methods, static analysis, code reviews, etc.
- **ISTQB®** Notation: **Static** Testing vs. **Dynamic** Testing.

Software Faults, Errors, Failure

- It all begins with a **mistake/error** (usually from a human).
- **Fault:** The **static** defect in the software
- **Error:** An incorrect **internal state** that is the manifestation of some fault
- **Failure:** External, **incorrect behaviour** with respect to the requirements or other description of the **expected behaviour**. (Incorrect output, or crashes from exceptions)
- Some material **may distinguish** between incorrect output (failures) and exceptions (error). Use your best judgement. ☺

Software Faults, Errors, Failure

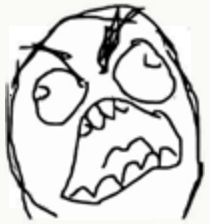
System Under Test (SUT)



Mistake/Error

Fault/Defect/Bug

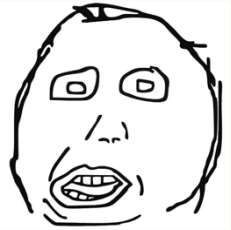
Software Faults, Errors, Failure



User

System Under Test (SUT)

Error (State)



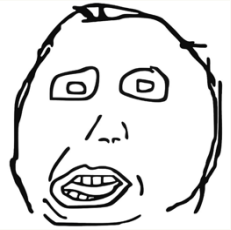
Mistake/Error

Fault/Defect/Bug

Software Faults, Errors, Failure

System Under Test (SUT)

Error (State)



Mistake/Error

Fault/Defect/Bug

Failure

Software Faults, Errors, Failures - Describe

```java
public static int countZeros(int[] numbers) {

    int zeroElements = 0;
    for (int i = 1; i < numbers.length; i++) {
        if(numbers[i] == 0) {
            zeroElements++;
        }
    }

    return zeroElements;
}
```

Software Faults, Errors, Failures - Describe

The error is that the algorithm will not check the first index of the array.

```
public static int countZeros(int

    int zeroElements = 0;
    for (int i = 1; i < numbers.length; i++) {
        if(numbers[i] == 0) {
            zeroElements++;
        }
    }
    return zeroElements;
}
```

This is the fault.

- i should start at 0!

Test 1: {1, 2, 0}

Expected Value: 1

Actual: 1

Test 1: {0, 2, 0}

Expected Value: 2

Actual: 1

This is the failure.

Does not match the expected result.

Exercise 2: Faults, Errors and Failures

- **Scenario 1**: A popular online shopping platform has a feature that allows users to add items to their cart and proceed to checkout. Recently, users reported that when they add an item to their cart and navigate away from the page, the item count displayed at the top right of the screen resets to zero, although the items are still present when they visit the cart.

- **Scenario 2:** A banking mobile app provides users with transaction history for the past month. Users recently reported that transaction dates are shown one day earlier than the actual transaction date. This issue appears when users view their transaction history, and the displayed dates do not match their expected transaction days, leading to confusion and concern about the app's reliability.

© Francisco Oliveira Neto - 2018

Exercise 2: Answers

Scenario 1:

- **Fault:** Some logic in the codebase causes the displayed item count to reset incorrectly.
- **Error:** When a user adds an item and navigates to a different page, the displayed count resets.
- **Failure:** Users see an incorrect cart count that is inconsistent with the number of items added.

Scenario 2:

- **Fault**: The code contains an incorrect time-zone offset logic that affects the display of transaction dates.
- **Error**: The incorrect date offset creates transactions with invalid state.
- **Failure**: The dates displayed to the user differ from the expected ones.

© Francisco Oliveira Neto - 2018